SAFE DEAL

Secure Escrow Web Platform

Sagat Alikhan ● Sauranbay Kanagat ● Salamat Nurila

Project Overview

Problem Analysis

- Online transactions lack trust
- Buyers pay, never receive the product, Sellers ship and never receive payment
- No trusted third party online
- No controlled interactions between users

Proposed Solutions

- SafeDeal holds funds in escrow
- Funds released only after delivery confirmed
- Dispute resolution system built-in
- Trust scores and ratings per user

FRONTEND REQUIREMENTS

Angular implementations

Services & Interfaces

Angular services with HttpClient wrap every API call; TypeScript interfaces model all response shapes

Routing

/landing, /dashboard, /deals, /deal/:id, /profile, /login -
Angular Router with guards on protected pages

4+ click events → API

Deal creation, payment, delivery confirm, dispute - each button triggers a live HTTP request

@for / @if Control flow

@for loops render deal lists; @if / @else handles loading states, empty states, role visibility

4+ Form Controls [(ngModel)]

Register, Login, New deal, Add Funds forms - all use two-way binding via Forms Module

CSS styling

Tailwind utility classes applied to every component for responsive, consistent UI

Auth & Communication

JWT Auth - Login, Logout, Interceptor

HTTP interceptor auto attaches Bearer token to every request; logout blacklists the refresh tokens.

1+Services using HttpClient

AuthService, DealService, TransactionService - all extend a base HttpClient wrapper

Graceful API Error handling

try/catch + catchError pipe display user-friendly messages in the UI on 4xx/5xx responses

BACKEND REQUIREMENTS

Models &relationships

User

balance,
escrow_balance,
trust_score

Deal → buyer/seller

Deal

status, price, title, description

FK → User

Transaction

amount, timestamp, type

FK → User, Deal

Rating

score, comment, created_at

FK → User, Deal

4 models - 3+ Foreign key relationships - Custom UserManager(AbstractUser)

Serializers

Serializer(base):
LoginSerializer - validates credentials
FundSerializer - validates deposit amount

ModelSerializer:
DealSerializer - full CRUD fields
RatingSerializer - nested user data

Views

Function based (FBV):
@api_view - pay(), confirm_delivery()
@api_view - open_dispute(), add_funds()

Class based (CBV):
DealListCreateView(APIView) - GET/POST
DealDetailView(APIView) - GET/PUT/DELETE

AUTH CORS CRUD

JWT Auth:
simplejwt login - logout (blacklist) - refresh
request.user links deals to owners

CORS:
django-cors-headers → localhost:4200
CRUD:
Deal - full GET/POST/PUT/DELETE